Speculative Sandboxes for Agentic Development

When to Fork, What to Verify, and When to Stop

Yossi Eliaz

Cambridge SRG seminar companion | 13 October 2026

Cambridge talk | Evidence-aware reduction

Abstract

A speculative sandbox is an executable continuation that may be discarded. Its value is neither start-up speed nor the number of parallel agents, but verified progress relative to resource cost. We separate state fidelity, private divergence, authority and irreversible effects from the economic question of whether to fork at all. From speculative decoding we borrow draft/verify separation, adaptive depth/width and acceptance budgeting—*not* exact-distribution preservation for arbitrary code. We derive correlated-success and marginal-coverage equations, distinguish verifier passes from correctness and independent evidence, and give a fail-closed controller protocol. An executable finite-world reference model tests admission, correlated replanning and receipt-bound promotion. Real sandbox integration, learned calibration and runtime gains remain **unmeasured proposals**.

1 The framing: speculate on execution, never on authority

A draft model proposes tokens and a target model checks them. Likewise a software controller may explore candidate changes while a separate verifier determines which exact artifact, if any, can advance. What transfers is prediction-driven allocation of speculative width, depth and checking effort. What does *not* transfer is the exact decoding theorem, cheap parallel token verification or free residual correction: failed patches need diagnosis, new executions and often new tests. This is speculative execution for agentic development, not a replacement for the build cache or a new language-model decoder.

2 First ask whether a fork should exist

A sandbox is an *execution/policy boundary*, not a synonym for a microVM or a process. A *forkable continuation* is a private branch of a declared reached state S_0. Its contents may include filesystem, environment and optionally live memory/processes; copied guest state does *not* copy the external world or publication authority. The Cambridge talk distinguishes named-snapshot *restore fan-out* from live-memory fork until the nonce/process-survival probe passes [1].

For a task class $\mathcal{T}$, the following are gates, not performance results:

Three distinct decisions. *Admissible* means state, privacy, authority and external-effect gates hold. *Forkable* additionally requires reproducible capture/restore semantics. *Advantageous* means that a fork strategy actually improves the specified utility against the best warm alternative. If any hard gate fails, do not fork; if the warm baseline wins, *choose it*. Performance cannot compensate for weak isolation.

Gate	Required evidence before claiming a useful fork
State	Name the bytes, runtime state and external dependencies needed by the next action; say explicitly whether memory survives.
Fidelity	On pinned tasks $t \in \mathcal{T}$, parent and restored continuation agree on required observations under controlled seeds/inputs and declared nondeterminism; record failures.
Privacy	Each child gets private writes and unique runtime identity/RNG; no child can modify another's evidence or the verifier.
Authority	Controller owns immutable check definitions, epoch, evidence ledger and promote permission; secrets/leases are reissued, never trusted because they were copied.
Effects	Network/API/billing consequences are fenced, simulated, or idempotently reconciled; restore never undoes outside effects.
Cost	Compare against an equivalent *warm cached* baseline, including capture, restore, divergence, verification and contention.

Table 1: Operational criteria; none follows from a fast create benchmark alone.

For K equivalent tasks, a simple *work-cost* accounting is

$$\begin{aligned} C_{\text{warm}} &= K(P_w + E + V), \\ C_{\text{fork}} &= P_0 + H + K(R + E + V) + D. \end{aligned} \tag{1}$$

Here P_w is warm preparation, P_0 preparing the parent, H capture, R restore, E execution, V verification and D coordination. The model assumes equivalent trials and unchanged E, V. Under this simplified additive model, fork saves work iff

$$\boxed{KP_w > P_0 + H + KR + D.} \tag{2}$$

For already-existing verified checkpoints, P_0, H are sunk for the *next* decision, but remain in complete system accounting. More generally, different execution modes can have different probabilities of success. A deadline-T controller compares $U_m = B\mathbb{P}_m(\text{verified by } T) - \lambda\mathbb{E}_m[C]$ for $m \in \{\text{warm}, \text{fork}\}$, subject to the same hard safety

gates. *No fork* is a first-class outcome. Tail latency and contention make wall time distinct from work cost [11]. H1 already specifies a warm-cache baseline, not a cold-start strawman [1].

3 What speculative decoding really proves

Let $q(x)$ be a cheap proposal distribution and $p(x)$ the exact target distribution over tokens. Draw $x \sim q$ and accept with probability

$$a(x) = \min\left(1, \frac{p(x)}{q(x)}\right), \qquad q(x) > 0. \tag{3}$$

Averaging over proposals gives *maximal-coupling* acceptance:

$$\begin{aligned} \alpha_{\text{token}} &= \sum_x q(x)a(x) = \sum_x \min\{p(x), q(x)\} \\ &= 1 - \underbrace{\frac{1}{2}\sum_x |p(x) - q(x)|}_{\text{TV}(p,q)}. \end{aligned} \tag{4}$$

Upon rejection, sample from $r(x) = [p(x) - q(x)]_+ / \text{TV}(p, q)$ when $\text{TV} > 0$. Then

$$\min(p, q) + \text{TV}(p, q)r = p, \tag{5}$$

pointwise. Thus the emitted token retains *exactly* the target distribution; no coupling can reuse a q-sample with a higher probability of equality [3]. The theorem depends on *known* p, q and this correction rule.

Never substitute a test pass for Eq. (4). For code, $A_i = \{\text{exact candidate } i \text{ passes trusted checks}\}$ and $a_i = \mathbb{P}(A_i \mid \text{history})$ predicts *verifier acceptance*. In general,

$$1 - \text{TV}(p, q) \neq \mathbb{P}(A_i) \neq \mathbb{P}(\text{semantically correct}_i). \tag{6}$$

An external verifier can still miss a bug; agent approval is not correctness. In particular, if $s = \mathbb{P}(A \mid C)$ is test sensitivity, $f = \mathbb{P}(A \mid C^c)$ is the false-pass rate, and $\pi = \mathbb{P}(C)$ is prior correctness, then

$$\mathbb{P}(C \mid A) = \frac{s\pi}{s\pi + f(1 - \pi)}, \tag{7}$$

when the denominator is positive. Passing a test warrants only the claims its coverage supports. The child never controls the checker.

3.1 A precise, but narrow, Gaussian bridge

For two normalized Gaussians $p = N(\mu_1, \Sigma)$ and $q = N(\mu_2, \Sigma)$ with the *same* positive definite covariance, the paper's two-factor weighted disagreement is

$$\Delta = \tfrac{1}{2}(\mu_1 - \mu_2)^T \Sigma^{-1}(\mu_1 - \mu_2). \tag{8}$$

Their TV overlap and KL divergence obey

$$\boxed{\alpha_{\text{token}} = 2\Phi\left(-\sqrt{\Delta/2}\right), \quad D_{\text{KL}}(p\|q) = \Delta.} \tag{9}$$

This identity requires *normalized, equal-covariance* Gaussian densities. It derives from the likelihood-ratio separating hyperplane and one-dimensional Gaussian tails. It is *not* a general mapping from the reducer's disagreement to patch correctness: equal means with unequal covariances yield $\Delta = 0$ but TV> 0; identical wrong patches can agree perfectly without adding evidence [2]. The Gaussian product integral in the earlier paper is not this overlap integral.

4 Acceptance-aware fork selection

Let A_i denote verifier acceptance and H encode fork ancestors, model/prompt/test hashes, failures and known shared inputs. In a fixed trial order, define

$$a_i^- = \mathbb{P}(A_i \mid H, A_1^c, \ldots, A_{i-1}^c). \tag{10}$$

The chain rule gives an *exact* correlated-success identity:

$$\boxed{\mathbb{P}\left(\bigcup_{i=1}^K A_i \,\middle|\, H\right) = 1 - \prod_{i=1}^K (1 - a_i^-).} \tag{11}$$

Replacing a_i^- by unconditional pass rates silently assumes independence. The history must retain observed failures and shared ancestry. Cancellation or timeout is *not* a verified failure and must not be substituted for A_i^c in this identity.

For a fixed set S of candidate branches let $F(S) = \mathbb{P}(\bigcup_{i \in S} A_i)$. A new worker j contributes

$$\boxed{F(S \cup \{j\}) - F(S) = \mathbb{P}\left(A_j \cap \bigcap_{i \in S} A_i^c\right).} \tag{12}$$

Diversity is an objective, not decoration. What matters is the probability that j succeeds *when the current workers fail*. Since the event on the right shrinks as S grows, F is monotone and submodular for a *fixed* joint distribution. Static equal-cost greedy selection has the classic $(1-1/e)$ approximation to optimal *fixed-budget coverage* [6]. The guarantee does *not* transfer automatically to adaptive policies or weighted cost/latency objectives; adaptive submodularity requires additional conditions [7].

4.1 Concrete correlation penalty

A transparent illustration takes a shared latent difficulty $Z \sim \text{Beta}(u, v)$ and conditionally IID branch outcomes $A_i \mid Z \sim \text{Bernoulli}(Z)$. Then

$$\begin{aligned} \mu = \frac{u}{u+v}, \quad \rho = \frac{1}{u+v+1}, \\ \mathbb{P}(\exists \text{ pass}) = 1 - \frac{(v)_K}{(u+v)_K}. \end{aligned} \tag{13}$$

With $(u, v) = (0.45, 1.05)$ and $K = 8$, each branch has marginal pass rate 0.30 and pairwise outcome correlation

0.40. The eight-branch success probability is *about 65.8%*, versus $1 - 0.7^8 \approx 94.2\%$ under independence. After four failures, the next same-family pass probability is $u/(u + v + 4) = 8.18\%$. These are **model calculations**, not measurements of islo or the factory.

4.2 Depth, width, and verification cost

For a dependent chain of D speculative edits, let L be the number of consecutive edits whose exact resulting artifacts pass, stopping at the first failure. The tail-sum formula is

$$\mathbb{E}[L] = \sum_{j=1}^{D} \mathbb{P}(A_1 \cap \cdots \cap A_j). \tag{14}$$

If conditional prefix survival is constant a, then $\mathbb{E}[L] = \sum_{j=1}^{D} a^j$. Unlike token decoding, a rejected code patch has no free residual-correction token: repair, rollback, rerun and verifier invocations all cost resources.

A controller can choose depth D, width W, model/prompt diversity, verifier order and cancellation policy to maximize a budgeted objective such as

$$\max_{\pi} \ \mathbb{P}_\pi(\text{verified success by } T) - \lambda \mathbb{E}_\pi[C], \tag{15}$$

subject to *unrelaxed* authority and final-artifact checks. Predictive acceptance must be calibrated on held-out tasks, with latency and resource use reported jointly. EAGLE-2 and OPT-Tree motivate context-dependent tree structure; EAGLE-3 improves the proposal mechanism itself. These are inspirations, not correctness theorems for software agents [4, 5, 8].

5 A runtime policy: admit, speculate, replan, stop

Static versus adaptive. Fixed-set $F(S)$ is submodular; after observed failures, a sequential controller estimates

$$p_j(h) = \mathbb{P}(A_j \mid h), \tag{16}$$

where h includes the immutable work order and completed outcomes. A simple *one-step heuristic* is

$$G_j(h) = B\, p_j(h) - \lambda c_j(h) - \eta \ell_j(h), \tag{17}$$

with incremental total compute/verification cost c_j and latency impact ℓ_j. If no admissible $G_j > 0$, stop or use warm execution. This is neither a globally optimal stopping theorem nor a production calibration model; portfolio-level parallel choices require conditional *marginal* value, not sum of standalone scores.

Which failures matter. Distinguish `planned`, `started`, `passed`, `failed`, `timed out`, and `cancelled`. Only a trustworthy observed failure updates the *success-conditioned-on-failure* probability in Eq. (11); absence of an outcome is censored information. When sibling failures reveal common causes, try a different hypothesis/model/fixture instead of paying for indistinguishable clones. Test the prediction on held-out workloads, not on duplicated forks of the same task.

Issue-time effects. A speculative API call can disclose data or incur a charge *before* the branch is selected; rollback cannot unsend it. This is the ghost-tool-call issue [9]. Fence or synthesize observable tool calls before dispatch. Separately, Sherlock studies overlap of downstream agent work with verification and rollback, but its reported benefits are not forkable-sandbox measurements [10].

6 Acceptance is not independent evidence

In the paper's independent Gaussian common-target model, worker i returns estimate $\hat{\theta}_i$, information P_i, evidence IDs and lineage. The current reducer forms $P = \sum_i P_i$ and $q = \sum_i P_i \hat{\theta}_i$, rejects declared duplicate IDs and transports ancestry. It *does not* infer covariance from lineage or implement selection correction [2].

If estimates share latent error U and have independent noise ε_i, write $Y_i = \theta + U + \varepsilon_i$ with variances τ^2 and σ^2. Then

$$\mathrm{Var}(\bar{Y}) = \tau^2 + \frac{\sigma^2}{K}, \quad K_{\text{eff}} = \frac{K}{1 + (K-1)\rho}, \tag{18}$$

where $\rho = \tau^2/(\tau^2+\sigma^2)$. Eight estimates with $\rho = 0.4$ have $K_{\text{eff}} \approx 2.11$ under this model. *Acceptance correlation* and *estimation-error correlation* are separate quantities; this effective sample size is not a success-probability shortcut.

With a *justified* covariance matrix C and a common-target observation map H_0, generalized least squares gives

$$\begin{aligned} \hat{\theta} &= (H_0^T C^{-1} H_0)^{-1} H_0^T C^{-1} Y, \\ \mathrm{Cov}(\hat{\theta}) &= (H_0^T C^{-1} H_0)^{-1}. \end{aligned} \tag{19}$$

A candidate fork-DAG covariance model is $C_{ij} = \sum_{a \in \mathrm{Anc}(i) \cap \mathrm{Anc}(j)} T_a + \mathbf{1}_{i=j} R_i$. Ancestry alone does *not* identify these variance components: fit/validate them externally or **abstain** from claiming increased precision.

Selection creates another bias. Suppose 100 IID $N(\theta, 1)$ estimates each report $[Y_i - 1.96, Y_i + 1.96]$. If we choose the largest estimate, coverage becomes

$$\mathbb{P}(\theta \in I_{\arg\max Y_i}) = \Phi(1.96)^{100} - \Phi(-1.96)^{100} \approx 7.95\%. \tag{20}$$

So even initially valid 95% intervals may become invalid after choosing winners. A valid reducer must record *rejections and selection history*, then use simultaneous/selection-aware or held-out inference; unchanged marginal intervals are not automatically valid after search. This is a **synthetic calculation**, not measured agent behavior.

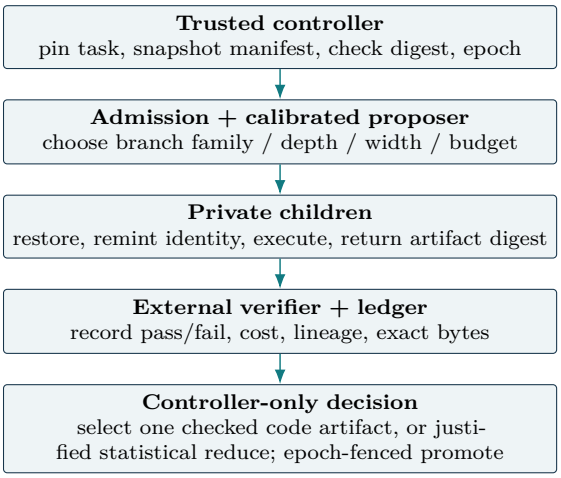


Figure 1: Proposed protocol. Only the controller can issue the final promote operation.

7 A minimal end-to-end controller

Admission. An immutable work-order key binds repository tree, initial-state digest, tool/model/prompt versions, task and selection context. Retried evaluation of *the same observation* reuses its evidence token; new samples get new tokens, but keep common ancestry. Do not count retries or speculative replicas twice.

Execution. Keep warm compiler/dependency state where safe, while child writes remain private; reseed uniqueness and remint authority outside copied guest state. Cheap tests may reject drafts early but cannot replace the sealed final verifier. Cancel losers only after their outcome/selection-accounting obligations are satisfied; mark cancelled outcomes *unobserved*, not failed.

Verification. An external checker binds `(task/input, artifact bytes, verifier digest, evaluation ID, epoch, verdict)` to a controller-authenticated receipt. Persist both passes and negative/censored outcomes. A *green* receipt asserts the specified checks ran on these bytes; it does not prove general program correctness. Different result types have different fan-in: select one patch, recheck any composed patch, or pool common-target measurements only with justified covariance.

Publication. Persist exact artifact bytes and checks before teardown; compare current epoch/version at a single controller-owned compare-and-swap. Stale workers can finish but cannot publish. Untrusted children must never write scores, sealed tests, receipts or promote tokens. The seminar's serial factory records motivate these obligations but do not demonstrate the complete forked protocol [1].

8 Executable toy and mathematical audit

An executable finite-world simulator and two standard-library test suites accompany this note. The *finite-world model* defines explicit, hypothetical joint pass events; its probabilities are known by construction, *not fitted* from real agents. It demonstrates all-or-nothing admission gates, a warm-cache win, a perfect-duplicate sibling worth no extra coverage, posterior replanning after verified failures, stopping when expected value is negative, and distinct timeout/cancellation semantics.

The *controller mock* persists candidate bytes, executes an externally held predicate and authenticates a digest/epoch/evaluation-ID receipt before one promotion. Tests deliberately try altered artifacts, altered receipts, repeated evaluation tokens, stale epochs and an *incomplete checker that passes bad code*. The last case makes the distinction between a validated protocol and semantic correctness explicit. HMAC, in-memory persistence and the finite-world oracle are demonstration choices; no isolation, robust key service, durable atomic transaction or actual microVM is implemented.

Independent checks recompute TV residual mass, Gaussian overlap by numerical quadrature, conditional chain probabilities, submodularity, beta-binomial success, expected accepted-prefix length, generalized least squares, post-selection coverage and the cost identity.

9 Falsifiable evaluation, not a speedup claim

Preserve preregistration. The Cambridge `PREREGISTRATION.md` already fixes Gate 0 (memory nonce), fidelity, H1 warm-cache cost and H2 sibling-vs-stranger correlation, with predefined tests and rejection thresholds. Do *not* change those analyses or imply they have run. The following is a separately labelled **exploratory Phase D proposal**.

1. **Same tasks, same authority:** pin repositories, work orders, checker images/digests, snapshot semantics and compute/time budgets; compare serial warm, fixed-width fan-out, calibrated acceptance-only, and lineage-aware adaptive scheduling.
2. **Predict before seeing the outcome:** train conditional pass models on disjoint historical tasks; freeze and calibrate them; record predictive log loss, Brier score and reliability by fork family, failure class and workload difficulty.
3. **Measure complete cost:** prepare/capture/restore, agent calls, verifier executions, cancellation, external effects, queueing, memory and p50/p95 time-to-*first exact verified artifact*; count CPU-seconds and dollars per success. Include no-fork/warm-cache controls.
4. **Try hard negative controls:** cloned wrong assumptions, independent model/prompt families, the `CellBusy` path without its direct deterministic test, stale epochs, duplicate evidence, tampered oracle, speculative observable API calls and post-selection intervals.
5. **Report falsifiers:** adaptive speculation must beat the best *warm* mode on cost/verified-progress or deadline

performance; lineage complexity is warranted only by better held-out conditional predictions or defensible coverage. Report null, worse, timeout and verifier false-pass outcomes, not only winning attempts.

Talk placement. A 90-second optional Q&A bridge after the seminar's runtime/evidence boundary: *"A fork is worthwhile only if it preserves useful state, beats the warm alternative on verified progress per budget, and cannot grade or publish itself. Speculation then asks: what is the probability this* next *branch succeeds* when the existing branches fail*? That probability controls the scheduler, while independent evidence controls statistical confidence."*

Takeaway

Fork only when admissible and better than the warm alternative.
Speculate on candidates; externally verify exact bytes; replan after negative evidence; promote once. Count statistical confidence by provenance and calibrated dependence, never by green-check or worker counts.

References

[1] Y. Eliaz, *Forkable Sandboxes Cambridge Seminar*, 2026. Claim fence and preregistration in the repository.

[2] Y. Eliaz, *Evidence-Aware Reduction for Forkable Compute*, arXiv:2607.09689v4, 2026.

[3] Y. Leviathan, M. Kalman, Y. Matias, *Fast Inference from Transformers via Speculative Decoding*, ICML, 2023.

[4] Y. Li et al., *EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees*, EMNLP, 2024.

[5] Y. Li et al., *EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test*, NeurIPS, 2025.

[6] G. L. Nemhauser, L. A. Wolsey, M. L. Fisher, *An Analysis of Approximations for Maximizing Submodular Set Functions—I*, Mathematical Programming 14, 1978.

[7] D. Golovin, A. Krause, *Adaptive Submodularity: Theory and Applications in Active Learning and Stochastic Optimization*, JAIR, 2011.

[8] J. Wang et al., *OPT-Tree: Speculative Decoding with Adaptive Draft Tree Structure*, TACL 13, 2025.

[9] B. Mohammadi et al., *Ghost Tool Calls: Issue-Time Privacy for Speculative Agent Tools*, arXiv:2606.02483, 2026.

[10] Y. Ro et al., *Sherlock: Reliable and Efficient Workflow Execution*, 2025.

[11] J. Dean, L. A. Barroso, *The Tail at Scale*, CACM 56(2), 2013.